HACKNEX — COMPLETE SOFTWARE REQUIREMENTS SPECIFICATION

Project Name: HackNEX
Organizer: NEXUS Club
Institution: Karunya Institute of Technology and Sciences
Location: Coimbatore, Tamil Nadu, India
Event Mode: Online
Event Dates: October 7–9, 2026
Registration Period: September 8–October 1, 2026
Team Size: Exactly 4 participants
Expected Participants: ~1,500
Expected Teams: ~400 teams at full four-member capacity
Document Version: 2.0
Document Type: Software Requirements Specification
Status: Master Requirements Baseline

---

1. DOCUMENT PURPOSE

This document defines the complete functional, technical, operational, security, performance, and administrative requirements for the HackNEX hackathon website.

It will serve as the primary reference for:

- UI/UX design
- Frontend development
- Backend development
- Database development
- Authentication
- Registration
- Payment integration
- Email communication
- Media management
- Admin panel
- Testing
- Deployment
- Maintenance

Any feature not defined in this document should be considered **outside the initial implementation scope unless explicitly approved later**.

---

2. PROJECT OVERVIEW

HackNEX is an online hackathon organized by **NEXUS Club of Karunya Institute of Technology and Sciences**.

The website will function as the central digital platform for HackNEX.

Its primary responsibilities are:

1. Presenting hackathon information.
2. Allowing visitors to explore the event.
3. Providing a premium landing experience.
4. Allowing users to create accounts.
5. Verifying user email addresses.
6. Allowing authenticated users to register a team.
7. Allowing a captain to register exactly four participants.
8. Collecting participant information.
9. Handling registration payment through the designated Karunya payment mechanism.
10. Verifying payment.
11. Confirming successful registration.
12. Sending confirmation and event information through email.
13. Providing administrators with registration and payment management tools.
14. Managing website content and media.
15. Supporting approximately 1,500 participants.

---

3. PROJECT VISION

The HackNEX website should not look like a conventional college registration portal.

It should provide a:

> **Modern, premium, futuristic and technology-focused hackathon experience.**

The website should communicate:

- Innovation
- Technology
- Competition
- Creativity
- Community
- Professionalism

---

4. PROBLEM STATEMENT

Traditional/manual hackathon registration can result in:

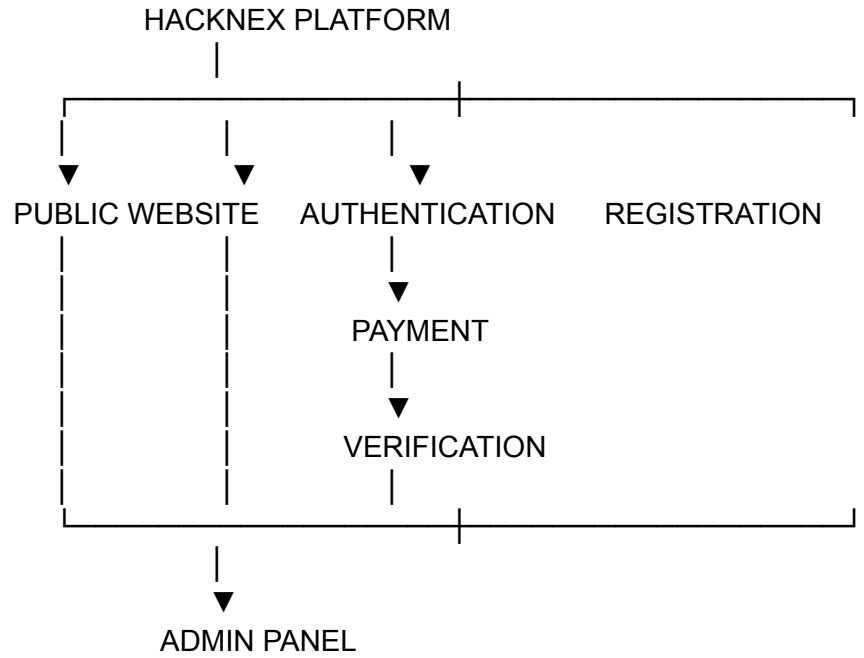
- Duplicate registrations
- Incorrect participant information
- Incomplete team information
- Payment mismatches
- Difficulty verifying payments
- Difficulty maintaining participant records
- Manual communication
- Administrative workload
- Poor participant experience

HackNEX requires a centralized platform that solves these problems.

---

5. PROPOSED SOLUTION

The proposed system consists of four major components:



---

6. PROJECT OBJECTIVES

The system shall:

1. Provide a professional HackNEX website.
2. Provide all required hackathon information.
3. Provide responsive desktop and mobile experiences.
4. Provide secure user authentication.
5. Require email verification.
6. Prevent unverified users from registering.
7. Allow one captain to register an entire four-member team.
8. Collect complete information for all four members.
9. Collect food preferences for all members.
10. Support participants from Karunya and external institutions.
11. Use the designated Karunya payment mechanism.
12. Verify payment before confirming registration.
13. Send confirmation only after successful payment verification.
14. Prevent duplicate team/participant registrations.
15. Provide an administrative management system.
16. Provide image/media management using Cloudinary.
17. Support approximately 1,500 participants.
18. Protect participant data.
19. Provide reliable email communication.
20. Maintain accurate payment and registration records.

---

7. PROJECT SCOPE

7.1 Included

Public Website

- Loading screen
- Home
- About
- Domains
- Details
- Schedule
- Prizes
- Sponsors
- Host/Institution
- FAQ
- Registration CTA
- Footer
- Image gallery/media

Authentication

- Signup
- Login
- Email verification
- Forgot password
- Password reset
- Logout
- Session management

Registration

- Team registration
- Captain registration
- Three additional members
- Participant information
- Food preference
- Review
- Registration ID

Payment

- Karunya payment mechanism
- Payment initiation
- Payment status
- Payment verification
- Payment failure handling
- Payment reconciliation

Communication

- Verification email
- Password reset
- Registration confirmation
- Important event announcements

Administration

- Dashboard
- Team management
- Participant management
- Payment management
- Content management
- Media management
- Search
- Filtering
- Export

---

8. OUT OF SCOPE FOR INITIAL RELEASE

The following are not required for Version 1:

- Participant social network
- Participant chat
- Team chat
- Project submission
- Judge portal
- Mentor portal
- Live leaderboard
- Certificate generation
- Automated judging
- In-app messaging
- Real-time event collaboration

These can be considered for future versions.

---

9. STAKEHOLDERS

Stakeholder	Responsibility
NEXUS Club	Event organization
Organizing Committee	Event administration
Karunya Institute	Institutional support
Finance Team	Payment management
Technical Team	Platform development
Participants	Hackathon participation
Team Captains	Team registration
Sponsors	Sponsorship
Administrators	Website management

---

10. USER ROLES

10.1 Visitor

Can:

- View public website
- View event information
- View domains
- View schedule
- View prizes
- View sponsors
- View FAQ
- View gallery
- Signup
- Login

10.2 Registered User / Captain

Can:

- Login
- Verify email
- Register a team
- Add three team members
- Enter participant information
- Select food preferences
- Review registration
- Proceed to payment
- View registration status
- Receive confirmation

The captain is the primary registration account.

10.3 Team Member

Team members:

- Do not need separate website accounts for initial registration.
- Are registered by the captain.

- Have their information stored as participant records.

10.4 Admin

Can:

- View dashboard
- View teams
- View participants
- View payments
- Manage content
- Manage gallery/media
- Search records
- Filter records
- Export records

10.5 Super Admin

Optional elevated role.

Can:

- Manage administrators
- Modify system configuration
- Access audit logs
- Perform sensitive operations

---

11. HACKATHON INFORMATION

The website must display:

- HackNEX name
- Official tagline
- Theme
- Organizer
- Institution
- Event mode
- Event dates

- Registration dates
- Team size
- Registration fee
- Eligibility
- Rules
- Domains
- Schedule
- Prizes
- Sponsors
- FAQ
- Contact information

Current confirmed information:

Name: HackNEX
Organizer: NEXUS Club
Institution: Karunya Institute of Technology and Sciences
Mode: Online
Dates: October 7–9, 2026
Registration: September 8–October 1, 2026
Team Size: Exactly 4

---

12. ELIGIBILITY REQUIREMENTS

- Eligible year(s) of study (UG , PG ,PHD)
- Eligible departments
- Inter-college participation
- Inter-department participation
- Geographic restrictions

These rules must appear on the website and be enforced in registration where appropriate.

---

13. TEAM REQUIREMENTS

Each team must contain exactly:

4 participants

Structure:

Team
│
├── Captain
├── Member 2
├── Member 3
└── Member 4

The system must not allow:

- Teams with fewer than four members.
- Teams with more than four members.

The system should also prevent a participant from being registered in multiple teams if that is the official HackNEX rule.

---

14. PARTICIPANT INFORMATION

Information required for **each participant**:

Field	Required
Full Name	Yes
Email	Yes
Phone Number	Yes
College	Yes
Department	Yes
Year of Study	Yes
LinkedIn URL	Yes / Final confirmation
Food Preference	Yes

Food preference:

- Vegetarian
- Non-Vegetarian

The captain selects food preference for:

- Captain
- Member 2

- Member 3
- Member 4

---

15. AUTHENTICATION REQUIREMENTS

15.1 Signup

Required:

- Name
- Email
- Password
- Confirm password

Process:

Signup
↓
Validate
↓
Create Account
↓
Send Verification Email
↓
User Verifies Email
↓
Account Activated

---

16. EMAIL VERIFICATION

Users must verify their email before accessing registration.

Unverified users attempting to register should receive a message such as:

> Please verify your email address before registering for HackNEX.

Verification links should:

- Expire after a defined period.
- Be single-use.
- Support resend functionality.
- Not expose sensitive information.

---

17. LOGIN

Login requires:

- Email
- Password

Successful login creates a secure authenticated session.

---

18. PASSWORD RESET

Users should be able to:

1. Select Forgot Password.
2. Enter registered email.
3. Receive reset email.
4. Open reset link.
5. Create new password.
6. Login again.

---

19. REGISTRATION PROCESS

The registration system should be implemented as a multi-step form.

Recommended structure:

Step 1
Team Information

Step 2
Captain

Step 3
Member 2

Step 4
Member 3

Step 5
Member 4

Step 6
Review

Step 7
Payment

Step 8
Confirmation

---

20. TEAM INFORMATION

The registration form should collect:

- Team name
- Captain designation

The system should determine whether team names must be unique.

Recommended:

> Team names should be unique.

---

21. CAPTAIN INFORMATION

Fields:

- Name
- Email
- Phone
- College
- Department
- Year of Study
- LinkedIn
- Food preference

The captain's account information can be prefilled into the registration form where appropriate.

---

22. MEMBER INFORMATION

For each of the remaining three participants:

- Name
- Email
- Phone
- College
- Department
- Year
- LinkedIn
- Food preference

---

23. REGISTRATION REVIEW

Before submission, the captain should see a complete summary.

Example:

TEAM
HackNEX Warriors

CAPTAIN
Name
College
Department
Year
Food Preference

MEMBER 2
...

MEMBER 3
...

MEMBER 4
...

PAYMENT
Registration Fee
No Refund Policy

The user should be able to edit information before payment.

---

24. REGISTRATION VALIDATION

The backend must validate:

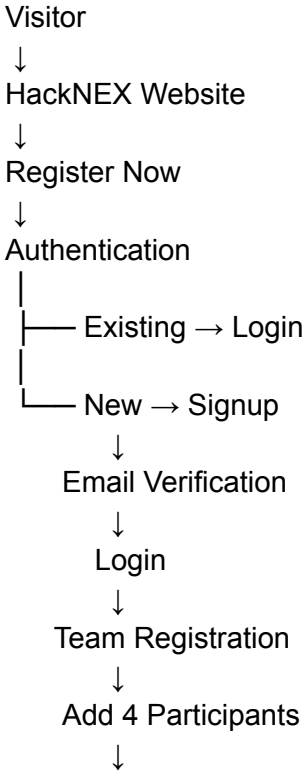
- Required fields
- Email format
- Phone format
- LinkedIn format
- Year
- Food preference
- Team size
- Duplicate participant
- Duplicate team
- Authentication
- Registration deadline

Frontend validation alone is insufficient.

Fastify's schema-based validation is well suited to enforcing API input constraints.

---

25. REGISTRATION WORKFLOW



Review Details
↓
Accept No-Refund Policy
↓
Payment
↓
Karunya Payment System
↓
Payment Verification
↓
Registration Confirmed
↓
Registration ID Generated
↓
Confirmation Email

---

26. PAYMENT REQUIREMENTS

Payment is a mandatory stage of registration.

The designated payment system is:

Karunya Payment System

It should be usable by:

- Karunya teams
- External college teams

The exact technical integration must be obtained from the Karunya authorities.

---

27. PAYMENT MODEL

The intended model is:

One Team
↓
Four Participants
↓
One Captain
↓
One Registration
↓

---

28. PAYMENT STATUS

Possible statuses:

PENDING
INITIATED
PROCESSING
SUCCESS
VERIFICATION_PENDING
VERIFIED
FAILED
CANCELLED
DUPLICATE

The exact state model should be finalized during backend design.

---

29. PAYMENT VERIFICATION

The system must distinguish between:

Payment completed

and:

Payment verified

Registration confirmation should occur only after the payment is successfully verified.

Potential verification mechanisms:

- Payment API
- Webhook
- Transaction ID
- Payment reference
- Manual admin verification

The exact mechanism depends on Karunya's payment infrastructure.

---

30. PAYMENT FAILURE

If payment fails:

Payment Failed
↓
Display Error
↓
Retry Payment
↓
Payment Verification

The system must avoid creating duplicate registrations when users retry.

---

31. PAYMENT DUPLICATION

The system must protect against:

- Double-clicking Pay
- Refreshing payment page
- Browser reopening
- Multiple payment attempts
- Duplicate transaction callbacks

Payment processing should be **idempotent** wherever possible.

---

32. NO REFUND POLICY

HackNEX will follow:

NO REFUND POLICY

This must be clearly displayed before payment.

The user should acknowledge the policy.

Example:

☐ I understand and agree that the
HackNEX registration fee is non-refundable.

The exact legal wording should be approved by the organizers.

---

33. REGISTRATION CONFIRMATION

The registration becomes confirmed only when:

Payment
↓
Successfully Completed
↓
Payment Verified
↓
Registration Confirmed

Then:

Registration ID
↓
Database
↓
Confirmation Email

---

34. REGISTRATION ID

Each confirmed team should receive a unique identifier.

Example:

HNX-2026-000001

The exact format can be finalized later.

The ID must be:

- Unique
- Permanent
- Searchable
- Associated with the team
- Associated with payment

---

35. EMAIL SYSTEM

Email is the primary post-registration communication mechanism.

Authentication

- Verification
- Password reset

Registration

- Registration received
- Payment status if required
- Registration confirmation

Event

- Event instructions
- Schedule updates
- Important announcements
- Changes
- Results/notifications if applicable

Resend provides an HTTPS-based email API and supports programmatic transactional email sending.

---

36. POST-REGISTRATION WEBSITE USAGE

After successful registration:

> Participants do not have any major additional workflow on the website in the initial version.

Further information should primarily be sent through:

> **Registered email**

This keeps the platform simple.

---

37. PUBLIC WEBSITE STRUCTURE

The website should contain:

Loading Screen
↓
Home
↓
About
↓
Domains
↓
Details
↓
Schedule
↓
Prizes
↓
Sponsors
↓
Host / Institution
↓
FAQ
↓
Registration CTA
↓
Footer

---

38. NAVBAR

Recommended desktop navigation:

HACKNEX Logo

Home
About
Domains
Details
Schedule
Prizes
Sponsors
Host
FAQ

[REGISTER NOW]

Requirements:

- Sticky navigation
- Active section indicator

- Smooth scrolling
- Mobile navigation
- Responsive layout
- Strong Register CTA

---

39. LOADING SCREEN

The website should initially display a HackNEX loading screen.

It may include:

- HackNEX logo
- NEXUS branding
- Animated typography
- Loading animation
- Progress indicator
- Short tagline

After loading:

Loading Screen
↓
Landing Page

The loading screen must not unnecessarily delay page interaction.

---

40. HOME / HERO SECTION

The hero section should include:

- HackNEX logo/name
- Tagline
- Event date
- Event mode
- Short description
- CTA
- Countdown
- Main visual

Example structure:

HACKNEX

Build. Innovate. Compete.

October 7–9, 2026

ONLINE HACKATHON

[REGISTER NOW]

Final wording is pending.

---

41. ABOUT SECTION

Should explain:

- What HackNEX is
- Why it is being conducted
- Who organizes it
- Objectives
- Participant experience
- Agenda

Potential flow:

Discover
↓
Ideate
↓
Build
↓
Collaborate
↓
Present
↓
Compete

---

42. DOMAIN SECTION

Domains will be represented using cards.

Each domain can contain:

- Name

- Icon
- Description
- Short explanation
- Optional technology tags

Mock domains may be used during development.

Final domains will be provided later.

---

43. DETAILS SECTION

This section will contain the official hackathon rules/details.

Potential subsections:

- Eligibility
- Team rules
- Participation rules
- Technology rules
- Submission requirements
- Judging criteria
- Code of conduct
- General instructions

The final document supplied by the organizers will be the authoritative source.

---

44. SCHEDULE SECTION

Schedule should use a visual timeline.

Each timeline entry can contain:

- Date
- Time
- Event
- Description
- Category

Example:

DAY 1
|

├── Opening Ceremony
├── Problem Statements
├── Team Formation
└── Hacking Begins

Actual schedule will be supplied later.

---

45. PRIZE SECTION

Display:

- First Prize
- Second Prize
- Third Prize
- Special Awards
- Sponsor Awards

Each prize can include:

- Rank
- Amount
- Description
- Additional benefits

---

46. SPONSORS SECTION

Sponsor cards should support:

- Logo
- Company name
- Sponsorship level
- Description
- Website link

Sponsor categories may include:

- Title Sponsor
- Gold Sponsor
- Silver Sponsor
- Technology Partner
- Community Partner

Final sponsors will be added later.

---

47. HOST / INSTITUTION SECTION

Information:

NEXUS Club

Karunya Institute of Technology and Sciences

Possible content:

- Institution description
- Logo
- Campus images
- Official website
- Organizer information

Although the event is online, this section establishes the institutional identity of the event.

---

48. FAQ SECTION

Initial FAQ:

What is HackNEX?

HackNEX is an online hackathon organized by NEXUS Club of Karunya Institute of Technology and Sciences.

When is HackNEX?

October 7–9, 2026.

Is the event online?

Yes.

How many members are required?

Exactly four participants per team.

Who registers the team?

The team captain.

Do all members need an account?

No. The captain registers all four participants.

Can students from other colleges participate?

Yes, subject to final eligibility rules.

How is payment made?

Through the designated Karunya payment system.

Is the registration fee refundable?

No.

When does registration close?

October 1, 2026.

How will further information be provided?

Primarily through the registered email.

Can registration details be edited?

The exact editing policy will be finalized.

---

49. FINAL CTA

A prominent CTA should appear near the end:

READY TO BUILD?

HACKNEX 2026

October 7–9

[REGISTER NOW]

---

50. FOOTER

Footer should contain:

HackNEX

- HackNEX branding
- NEXUS Club
- Karunya Institute

Quick Links

- Home
- About
- Domains
- Details
- Schedule
- Prizes
- Sponsors
- FAQ

Social

- NEXUS LinkedIn
- NEXUS Instagram

Institution

- Karunya official website

Legal

- Privacy Policy
- Terms & Conditions
- No Refund Policy

Copyright

---

51. IMAGE & MEDIA REQUIREMENTS

This is the **new section added for Cloudinary**.

The website is expected to contain approximately:

10–20 images initially

Potential images:

- Hero images
- NEXUS photos
- Hackathon/event photos
- KITS images
- Sponsor logos
- Gallery images

---

52. CLOUDINARY REQUIREMENT

Cloudinary will be used as the primary media storage and image-delivery service for dynamic website images.

Architecture:

Admin
↓
Upload Image
↓
Cloudinary
↓
Cloudinary URL
↓
Database Metadata
↓
React Website
↓
User

The actual image file should **not** be stored inside PostgreSQL.

The database should store metadata such as:

image_id
public_id
image_url
title
description
category

display_order
created_at
updated_at

---

53. IMAGE OPTIMIZATION

Cloudinary should be configured to:

- Automatically optimize image quality.
- Automatically select appropriate image formats.
- Resize images where appropriate.
- Serve responsive sizes.
- Deliver through CDN.

Cloudinary specifically recommends automatic quality and format selection (`q_auto` and `f_auto`) and responsive sizing for optimized delivery.

---

54. IMAGE CATEGORIES

Recommended categories:

hero
about
gallery
venue
sponsor
organizer
miscellaneous

---

55. GALLERY REQUIREMENTS

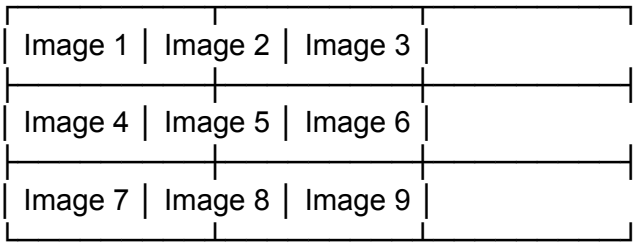
The website may contain a gallery section.

Gallery should support:

- Grid layout
- Responsive layout
- Image preview
- Lightbox/modal
- Lazy loading

- Optimized delivery
- Mobile support

For example:



---

56. ADMIN MEDIA MANAGEMENT

Admin should optionally be able to:

- Upload image
- Delete image
- Replace image
- Change title
- Change description
- Change category
- Change display order
- Enable/disable image

Example:

Gallery Manager

[Upload Image]

Image	Category	Status	Action
image01	Gallery	Active	Edit Delete
image02	Gallery	Active	Edit Delete
image03	Hero	Active	Edit Delete

---

57. IMAGE SECURITY

Admin-only image uploads must require:

- Authentication
- Authorization

- File type validation
- File size validation
- Upload restrictions

Only approved formats should be accepted, for example:

JPG
JPEG
PNG
WEBP

SVG uploads should be treated carefully because of potential security concerns.

---

58. IMAGE PERFORMANCE

The system should:

- Lazy-load non-critical images.
- Avoid loading original full-resolution assets when unnecessary.
- Use responsive image dimensions.
- Optimize images through Cloudinary.
- Use appropriate thumbnails.
- Preload only critical hero imagery.

Cloudinary's delivery architecture supports CDN-based delivery and responsive/optimized image delivery.

---

59. UI/UX REQUIREMENTS

The design should be:

- Modern
- Futuristic
- Premium
- Responsive
- Interactive
- Professional
- Consistent

---

60. ANIMATION REQUIREMENTS

Possible animations:

- Loading screen
- Hero entrance
- Scroll reveal
- Hover effects
- Card animation
- Timeline animation
- Gallery animation
- Registration success animation

Animations should not interfere with usability or accessibility.

---

61. RESPONSIVE DESIGN

Supported:

- Desktop
- Laptop
- Tablet
- Mobile

Special attention must be given to:

- Registration forms
- Navbar
- Payment flow
- Gallery
- Admin dashboard

---

62. ACCESSIBILITY

The system should support:

- Semantic HTML
- Keyboard navigation
- Accessible forms
- Screen readers
- Focus indicators

- Appropriate contrast
- Alternative text
- Reduced-motion preferences

---

63. ADMIN DASHBOARD

Dashboard should display:

Total Teams
Total Participants
Confirmed Teams
Pending Registrations
Verified Payments
Pending Payments
Failed Payments
Registrations Today

Optional:

Veg Participants
Non-Veg Participants
Colleges
Departments

These are especially useful for event logistics.

---

64. TEAM MANAGEMENT

Admin can:

- View teams
- Search teams
- Filter teams
- Open team details
- View members
- View captain
- View registration status
- View payment status

---

65. PARTICIPANT MANAGEMENT

Admin can:

- Search participants
- View participants
- Filter by college
- Filter by department
- Filter by year
- Filter by food preference
- View team
- View registration ID

---

66. PAYMENT MANAGEMENT

Admin can view:

- Registration ID
- Team name
- Captain
- Amount
- Payment ID
- Transaction reference
- Payment status
- Verification status
- Payment date

---

67. CONTENT MANAGEMENT

Admin may manage:

- Domains
- Schedule
- Prizes
- Sponsors
- FAQ
- Announcements
- Website images
- Gallery

---

68. DATA EXPORT

Admin should be able to export:

Team Data

- Registration ID
- Team name
- Captain
- Members
- Payment status

Participant Data

- Name
- Email
- Phone
- College
- Department
- Year
- LinkedIn
- Food preference
- Team

Payment Data

- Registration ID
- Transaction ID
- Amount
- Status
- Date

Preferred formats:

- CSV
- Excel

---

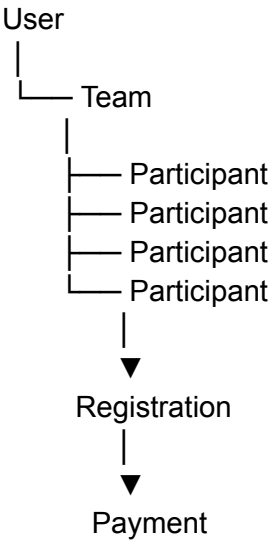
69. DATABASE REQUIREMENTS

Core entities:

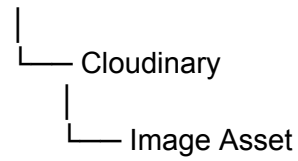
User
Team
Participant

Registration
Payment
Hackathon
Domain
Schedule
Prize
Sponsor
FAQ
Announcement
Media
Admin
AuditLog

Relationship:



Media:



---

70. DATABASE INTEGRITY

Database constraints should prevent:

- Duplicate users
- Duplicate registrations
- Invalid team sizes
- Invalid participant references

- Duplicate payment records
- Invalid payment states

Registration/payment operations should be transactionally safe.

---

71. API REQUIREMENTS

Authentication

POST /api/auth/signup
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/verify-email
POST /api/auth/resend-verification
POST /api/auth/forgot-password
POST /api/auth/reset-password

Public Content

GET /api/hackathon
GET /api/domains
GET /api/schedule
GET /api/prizes
GET /api/sponsors
GET /api/faqs
GET /api/media

Registration

POST /api/registrations
GET /api/registrations/:id
PUT /api/registrations/:id

Payment

POST /api/payments
GET /api/payments/:id
POST /api/payments/verify

Admin

GET /api/admin/dashboard
GET /api/admin/teams
GET /api/admin/participants

GET /api/admin/payments
GET /api/admin/media
GET /api/admin/export

---

72. SECURITY REQUIREMENTS

The application must implement:

- HTTPS
- Secure authentication
- Password hashing
- Email verification
- Secure sessions
- Role-based access control
- Backend authorization
- Input validation
- Rate limiting
- CSRF protection where applicable
- Secure cookies
- Security headers
- Environment variables
- Database access restrictions
- Audit logging

---

73. PAYMENT SECURITY

HackNEX must not store sensitive payment credentials.

The system must never store:

- Card numbers
- CVV
- UPI PIN
- Banking passwords

Payment credentials should remain within the approved payment system.

---

74. ADMIN SECURITY

Admin endpoints must be protected server-side.

This is not sufficient:

```
if (isAdmin) {
    showAdminPanel();
}
```

Instead:

Request
↓
Authentication
↓
Session Validation
↓
Role Verification
↓
Permission Check
↓
API Execution

---

75. PRIVACY REQUIREMENTS

Collected personal information includes:

- Name
- Email
- Phone
- College
- Department
- Year
- LinkedIn
- Food preference

The system must define:

- Purpose of collection
- Data usage
- Data access
- Data retention
- Data protection
- Privacy policy

---

76. PERFORMANCE REQUIREMENTS

The website should:

- Load quickly.
- Minimize JavaScript.
- Optimize images.
- Lazy-load non-critical assets.
- Cache static content.
- Optimize database queries.
- Paginate admin tables.

---

77. SCALABILITY

Expected:

> ~1,500 participants

Expected teams:

> ~375 teams

However, the architecture should comfortably accommodate at least:

> **3,000–5,000 participant records**

This provides a reasonable buffer.

---

78. TRAFFIC SPIKES

Special attention must be given to:

- Registration opening
- Registration deadline
- Payment periods
- Announcement periods

The system should handle concurrent:

- Logins
- Registration requests
- Database writes

- Payment callbacks
- Email jobs

---

79. EMAIL QUEUE

Emails should preferably be processed asynchronously.

Registration Confirmed
↓
Create Email Job
↓
Redis / BullMQ
↓
Email Worker
↓
Resend
↓
Participant

This prevents email sending from unnecessarily blocking the registration API.

---

80. ERROR HANDLING

Examples:

Existing Email

> An account with this email already exists.

Invalid Login

> Invalid email or password.

Unverified Account

> Please verify your email before continuing.

Duplicate Participant

> This participant is already registered.

Payment Failed

Payment could not be completed. Please try again.

Payment Pending

Your payment is being verified. Please wait.

Registration Closed

HackNEX registration is currently closed.

Server Error

Something went wrong. Please try again later.

---

81. EDGE CASES

The system must handle:

- Browser closing during registration
- Network interruption
- Page refresh
- Double-click submission
- Payment timeout
- Payment completed but callback delayed
- Payment completed but browser closed
- Duplicate payment
- Duplicate participant
- Duplicate team
- Expired verification link
- Multiple verification requests
- Registration deadline passing
- Email delivery failure
- Payment system unavailable
- Database failure
- Server restart
- Concurrent admin edits

---

82. SEO

Public website should support:

- Page title
- Meta description
- Open Graph
- Social sharing image
- Favicon
- Sitemap
- Robots.txt
- Canonical URLs
- Structured metadata

---

83. ANALYTICS

Optional analytics:

- Page views
- Registration CTA clicks
- Signup conversion
- Registration conversion
- Traffic source
- Device type
- Registration funnel

Analytics should avoid unnecessary collection of sensitive personal data.

---

84. TECHNICAL STACK

Frontend

React
TypeScript
Vite
Tailwind CSS
shadcn/ui
React Router
React Hook Form
Zod
Zustand
Framer Motion
Lucide React

Axios

---

85. BACKEND STACK

Node.js
TypeScript
Fastify
Prisma
PostgreSQL
Better Auth
Zod

Fastify is a suitable choice here because it provides TypeScript support and schema-based validation/serialization for API routes.

---

86. MEDIA STACK

Image Storage & Delivery

Cloudinary

Used for:

- Event photos
- Gallery
- Hero images
- Sponsor logos
- Venue images
- Organizer images

Cloudinary provides image optimization, transformations, responsive delivery and CDN-based delivery, making it appropriate for the website's media layer.

---

87. EMAIL STACK

Resend

Used for:

- Verification

- Password reset
- Registration confirmation
- Event announcements

---

88. CACHE / QUEUE

Redis
+
BullMQ

Used for:

- Email queues
- Background jobs
- Rate limiting
- Temporary state
- Caching where useful

---

89. PAYMENT

Karunya Payment System

Integration details remain pending.

---

90. TESTING

Unit

Vitest

E2E

Playwright

Test:

Signup
↓
Verification

↓
Login
↓
Registration
↓
Review
↓
Payment
↓
Verification
↓
Confirmation

---

91. API TESTING

Test:

- Authentication
- Authorization
- Registration
- Validation
- Payment
- Admin endpoints
- Media endpoints

---

92. LOAD TESTING

Test approximately:

100 concurrent users
250 concurrent users
500 concurrent users

Especially:

- Login
- Registration
- Payment
- API requests
- Database writes

---

93. DEPLOYMENT

Recommended:

Frontend
↓
Vercel

Backend
↓
Render / Railway

Database
↓
Neon / Supabase PostgreSQL

Redis
↓
Managed Redis / Upstash

Media
↓
Cloudinary

Email
↓
Resend

Monitoring
↓
Sentry

---

94. ENVIRONMENT VARIABLES

Examples:

DATABASE_URL
AUTH_SECRET
RESEND_API_KEY
REDIS_URL
CLOUDINARY_CLOUD_NAME
CLOUDINARY_API_KEY
CLOUDINARY_API_SECRET
PAYMENT_API_KEY

PAYMENT_SECRET
SENTRY_DSN

Secrets must never be committed to GitHub.

---

95. CLOUDINARY ENVIRONMENT SECURITY

Cloudinary credentials must be stored in environment variables.

Do not expose:

CLOUDINARY_API_SECRET

in frontend code.

Where possible, image uploads should use a secure server-generated upload signature or another controlled upload mechanism rather than exposing privileged credentials to the browser.

---

96. BACKUP REQUIREMENTS

Critical data:

- Users
- Teams
- Participants
- Registrations
- Payments

must be backed up.

Requirements:

- Automated database backups
- Defined retention period
- Recovery process
- Restricted backup access

---

97. MONITORING

Monitor:

- API errors
- Authentication failures
- Registration failures
- Payment errors
- Payment verification
- Email failures
- Database errors
- Server health
- Traffic spikes
- Cloudinary/upload failures

Sentry can be used for application error monitoring.

---

98. LOGGING

System logs should capture:

- Authentication events
- Registration events
- Payment events
- Admin actions
- API errors
- Email failures
- Important system events

Sensitive information should not be unnecessarily written into logs.

---

99. AUDIT LOGS

Admin-sensitive operations should be auditable.

Example:

Admin
↓
Edited Participant
↓

Timestamp
↓
Previous Value
↓
New Value

Potential events:

- Registration modification
- Payment status modification
- Team modification
- Participant modification
- Content update
- Media deletion
- Admin creation

---

100. ACCEPTANCE CRITERIA

Public Website

- Loading screen works.
- Home works.
- All sections work.
- Navigation works.
- Mobile layout works.
- Gallery works.
- Images load efficiently.
- Registration CTA works.

Authentication

- Signup works.
- Verification works.
- Login works.
- Logout works.
- Password reset works.

Registration

- Authentication required.
- Email verification required.
- Four members required.
- All required information validated.

- Food preferences captured.
- Duplicate participants prevented.
- Registration stored correctly.

Payment

- Karunya payment flow works.
- Payment status recorded.
- Payment verification works.
- Failed payments handled.
- Duplicate payments handled.
- No-refund policy displayed.

Confirmation

- Confirmation only after verified payment.
- Registration ID generated.
- Confirmation email sent.

Admin

- Dashboard works.
- Teams accessible.
- Participants accessible.
- Payments accessible.
- Content manageable.
- Media manageable.
- Data export works.

---

101. FUTURE ENHANCEMENTS

Future versions may add:

Participant Dashboard
Project Submission
Judge Dashboard
Mentor Dashboard
Live Leaderboard
Certificates
QR Check-in
Project Evaluation
Announcements Dashboard
Team Chat

Real-time Notifications
Advanced Analytics

---

102. PENDING INFORMATION

These must still be supplied by the organizers.

Event

- Official tagline
- Official theme
- Final event description

Eligibility

- Eligible years
- Eligible departments
- Eligible institutions
- Age restrictions
- Inter-college rules

Registration

- Exact registration fee
- LinkedIn requirement confirmation
- Team-name rules
- Editing rules
- Cancellation rules

Payment

- Karunya payment URL/system
- API availability
- Webhook availability
- Transaction verification mechanism
- External participant payment procedure
- Payment reconciliation process

Content

- Domain list
- Rules/details document

- Schedule
- Prize details
- Sponsor details
- FAQ final content

Branding

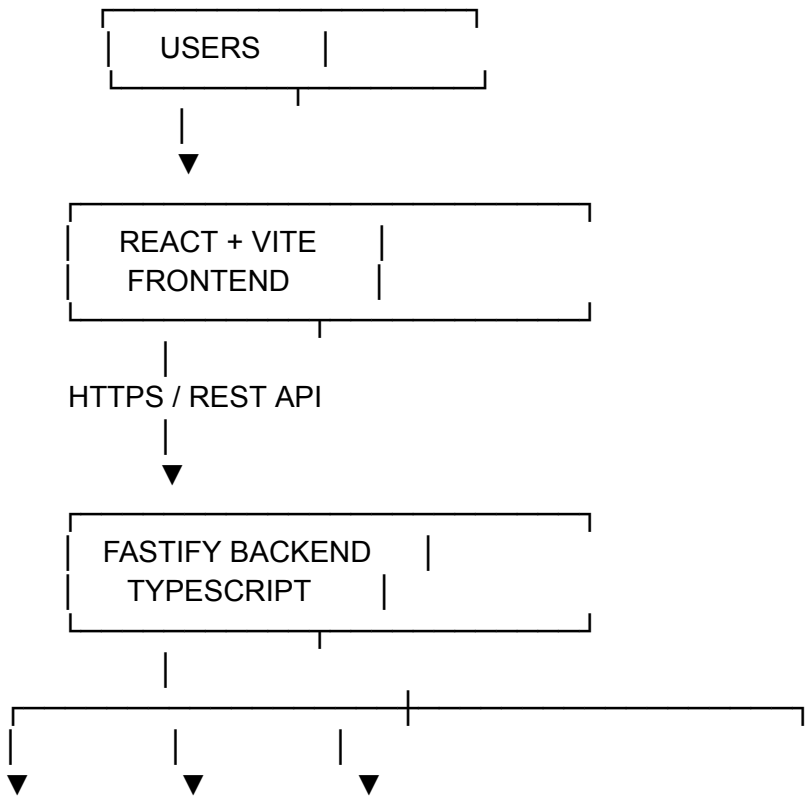
- HackNEX logo
- NEXUS logo
- Color palette
- Fonts
- Brand guidelines

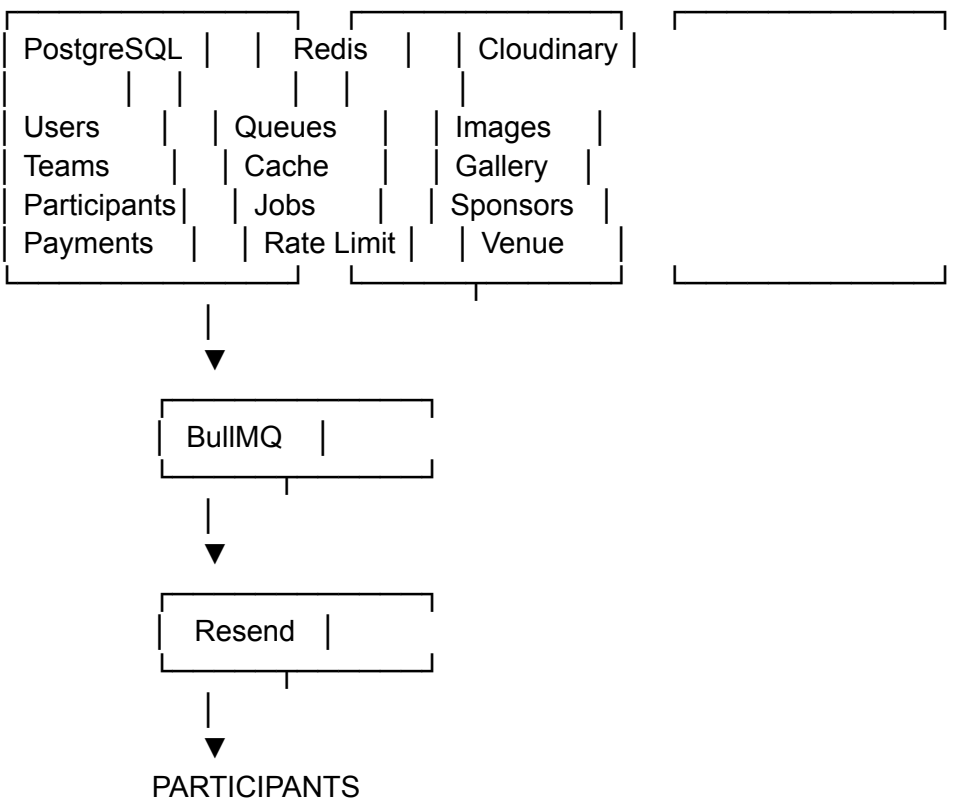
Social/External Links

- NEXUS LinkedIn
- NEXUS Instagram
- KITS official website
- Official HackNEX email

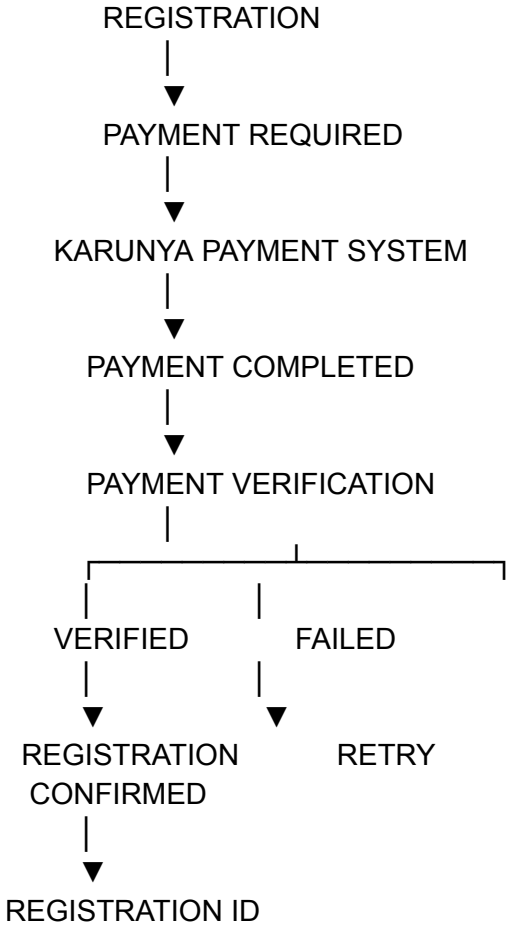
---

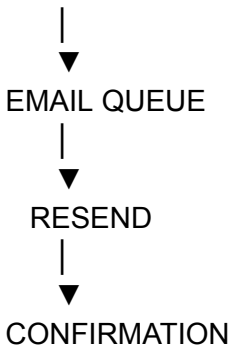
103. FINAL COMPLETE ARCHITECTURE




PostgreSQL
Redis
Cloudinary
Users
Teams
Participants
Payments
Queues
Cache
Jobs
Rate Limit
Images
Gallery
Sponsors
Venue
BullMQ
Resend
PARTICIPANTS

Payment:


REGISTRATION
PAYMENT REQUIRED
KARUNYA PAYMENT SYSTEM
PAYMENT COMPLETED
PAYMENT VERIFICATION
VERIFIED
FAILED
REGISTRATION CONFIRMED
RETRY
REGISTRATION ID



---

104. COMPLETE TECHNOLOGY LIST

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui
- React Router
- React Hook Form
- Zod
- Zustand
- Framer Motion
- Lucide React
- Axios

Backend

- Node.js
- TypeScript
- Fastify
- Prisma
- PostgreSQL
- Better Auth
- Zod

Media

- **Cloudinary**

Email

- **Resend**

Background Processing

- Redis
- BullMQ

Testing

- Vitest
- Playwright

Monitoring

- Sentry

Deployment

- Vercel
- Render/Railway
- Neon/Supabase
- Managed Redis

Development

- Git
- GitHub
- ESLint
- Prettier
- GitHub Actions

---

105. MASTER BUSINESS RULES

These are the most important rules in the entire system.

BR-01: Users must create an account or log in before registration.

BR-02: Email verification is required before registration.

BR-03: One captain registers the entire team.

BR-04: Every team must contain exactly four participants.

BR-05: Participant details must be collected for all four members.

BR-06: The captain selects food preference for every member.

BR-07: There is one registration payment per team.

BR-08: Karunya and external teams use the designated Karunya payment mechanism.

BR-09: Payment must be verified before registration is confirmed.

BR-10: Confirmation email is sent only after payment verification.

BR-11: HackNEX follows a no-refund policy.

BR-12: Participants primarily receive further event information through registered email.

BR-13: The platform must support approximately 1,500 participants.

BR-14: Administrative operations must be authorized on the backend.

BR-15: Payment credentials must not be stored by HackNEX.

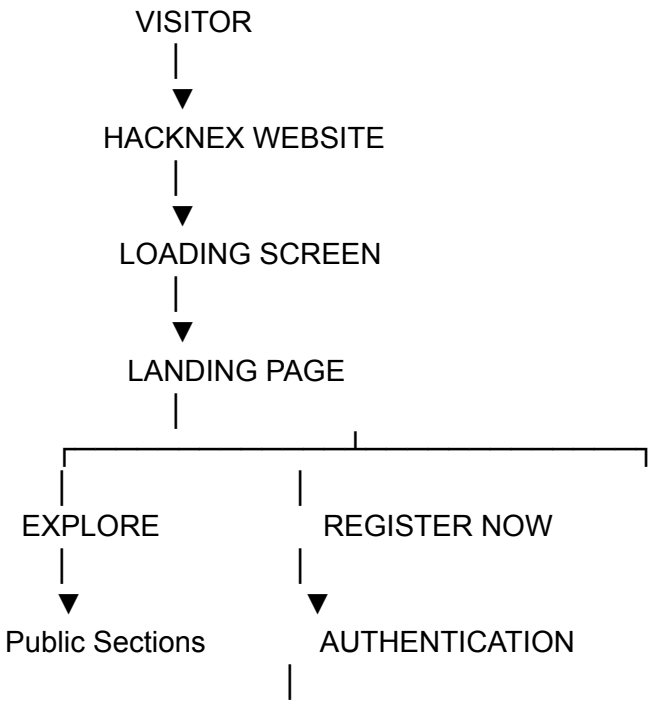
BR-16: Website images should be delivered through optimized media infrastructure.

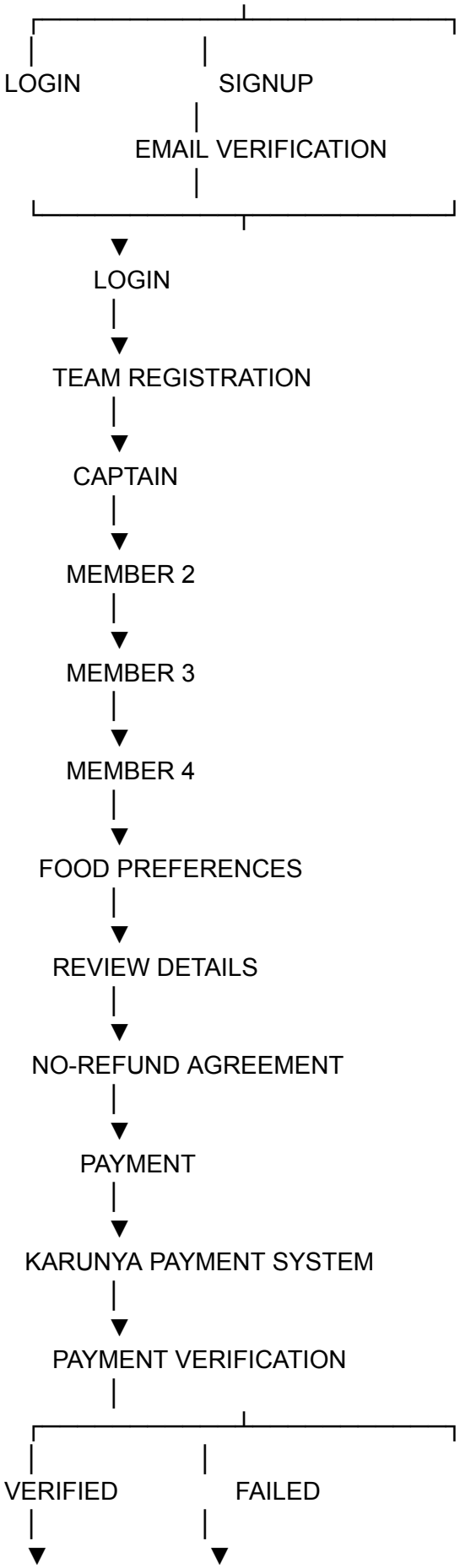
BR-17: Cloudinary stores/delivers website media; PostgreSQL stores only the relevant media metadata/URLs.

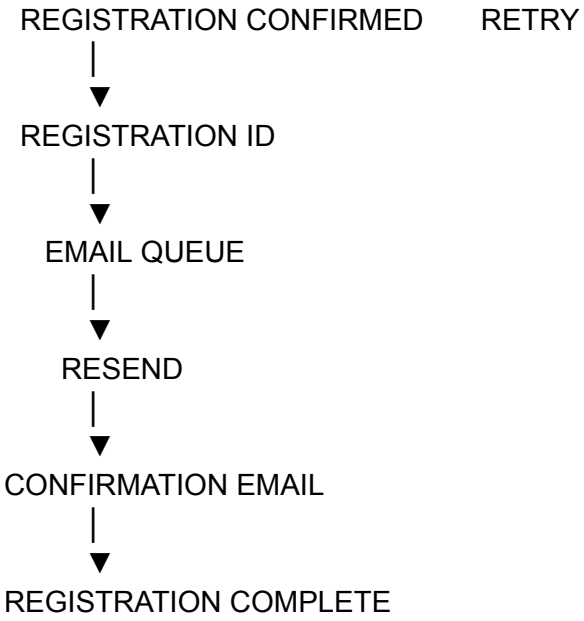
BR-18: Sensitive credentials must never be committed to the source repository.

---

106. FINAL END-TO-END SYSTEM




LOGIN
SIGNUP
EMAIL VERIFICATION
LOGIN
TEAM REGISTRATION
CAPTAIN
MEMBER 2
MEMBER 3
MEMBER 4
FOOD PREFERENCES
REVIEW DETAILS
NO-REFUND AGREEMENT
PAYMENT
KARUNYA PAYMENT SYSTEM
PAYMENT VERIFICATION
VERIFIED
FAILED


REGISTRATION CONFIRMED
RETRY
REGISTRATION ID
EMAIL QUEUE
RESEND
CONFIRMATION EMAIL
REGISTRATION COMPLETE